MULTITASKING :-

```
void StartTask02(void const * argument)
{
  for(;;)
  {
    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_10);
    osDelay(500);
  }
}
void StartTask03(void const * argument)
{
  for(;;)
  {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_11);
    osDelay(1000);
  }
}
```

SEMAPHORE :-

```
#include<string.h>
/* USER CODE BEGIN PV */
osSemaphoreId uartSemaphoreHandle;
/* USER CODE END PV */

void UART_Print(char *msg)
{
        HAL_UART_Transmit(&huart2, (uint8_t *)msg, strlen(msg), HAL_MAX_DELAY);
/* USER CODE END 0 */
}
```

```
/* USER CODE BEGIN 2 */
  osSemaphoreDef(uart_sem);

    uartSemaphoreHandle = osSemaphoreCreate(osSemaphore(uart_sem), 1);

    if (uartSemaphoreHandle == NULL)
    {
       Error_Handler();
    }
  /* USER CODE END 2 */

/* USER CODE END Header_StartTask02 */
void StartTask02(void const * argument)
{
  /* USER CODE BEGIN StartTask02 */
  /* Infinite loop */
  for(;;)
  {
            UART_Print("Task1 : Waiting for Semaphore... \r\n");

            if(osSemaphoreWait(uartSemaphoreHandle, osWaitForever)>=0){
                    UART_Print("Task1 : Semaphore Acquired...\r\n");

                    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);

                    osDelay(2000);

                    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

                    UART_Print("Task1 : Releasing Semaphore...\r\n");
```

```
                osSemaphoreRelease(uartSemaphoreHandle);
            }
            osDelay(1000);

  }
}

/* USER CODE END Header_StartTask03 */
void StartTask03(void const * argument)
{
  /* USER CODE BEGIN StartTask03 */
  /* Infinite loop */
          for(;;)
           {
                UART_Print("Task2 : Waiting for Semaphore... \r\n");

                if(osSemaphoreWait(uartSemaphoreHandle, osWaitForever)>=0){
                        UART_Print("Task2 : Semaphore Acquired...\r\n");

                        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_10, GPIO_PIN_SET);

                        osDelay(2000);

                        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_10, GPIO_PIN_RESET);

                        UART_Print("Task2 : Releasing Semaphore...\r\n");

                        osSemaphoreRelease(uartSemaphoreHandle);
                }
                osDelay(1000);

            }
```

```
/* USER CODE END StartTask03 */
}
```

MAILBOX:-

```
#include "FreeRTOS.h"
#include "queue.h"
#include <string.h>
QueueHandle_t mailbox;

mailbox = xQueueCreate(1,sizeof(char[50]));
 if(mailbox==NULL)
 {
        Error_Handler();
 }

void StartTask02(void const * argument)
{
  char tx_msg[50];

  for(;;)
  {
    strcpy(tx_msg, "Hello from sender\r\n");

    if(xQueueSend(mailbox, tx_msg, portMAX_DELAY) == pdPASS)
    {
      char status[] = "TASK02 (Sender) is RUNNING\r\n";
      HAL_UART_Transmit(&huart2, (uint8_t *)status,
                strlen(status), HAL_MAX_DELAY);
    }

    osDelay(1000);
```

```
    }
}

void StartTask03(void const * argument)
{
    char rx_msg[50];
    char msg[] = "Received: ";
    char status[] = "TASK03 (Receiver) is RUNNING\r\n";

    for(;;)
    {
        if(xQueueReceive(mailbox, rx_msg, portMAX_DELAY) == pdPASS)
        {
            HAL_UART_Transmit(&huart2, (uint8_t *)status,
                    strlen(status), HAL_MAX_DELAY);

            HAL_UART_Transmit(&huart2, (uint8_t *)msg,
                    strlen(msg), HAL_MAX_DELAY);

            HAL_UART_Transmit(&huart2, (uint8_t *)rx_msg,
                    strlen(rx_msg), HAL_MAX_DELAY);
        }
    }
}
```

QUEUE:-

```
#include "FreeRTOS.h"
#include "queue.h"
#include <string.h>
QueueHandle_t myQueue;
```

```
myQueue = xQueueCreate(5, sizeof(char[50]));

  if(myQueue == NULL)
  {
    Error_Handler();
  }

void Send(void const * argument)
{
  char tx_msg[50];

  for(;;)
  {
    strcpy(tx_msg, "Hello from Sender1\r\n");

    if(xQueueSend(myQueue, tx_msg,
              portMAX_DELAY) == pdPASS)
    {
      char status[] =
            "Sender1: Message sent\r\n";

      HAL_UART_Transmit(&huart2,
                (uint8_t *)status,
                strlen(status),
                HAL_MAX_DELAY);
    }

    osDelay(1000);
  }
}
void Send2(void const * argument)
```

```
{
   char tx_msg[50];

   for(;;)
   {
      strcpy(tx_msg, "Hello from Sender2\r\n");

      if(xQueueSend(myQueue, tx_msg,
                portMAX_DELAY) == pdPASS)
      {
         char status[] =
               "Sender2: Message sent\r\n";

         HAL_UART_Transmit(&huart2,
                     (uint8_t *)status,
                     strlen(status),
                     HAL_MAX_DELAY);
      }

      osDelay(1500);
   }
}
void Receive(void const * argument)
{
   char rx_msg[50];

   for(;;)
   {
      if(xQueueReceive(myQueue, rx_msg,
                  portMAX_DELAY) == pdPASS)
      {
```

```
        char status[] =
              "Receiver: Message received\r\n";

        HAL_UART_Transmit(&huart2,
                    (uint8_t *)status,
                    strlen(status),
                    HAL_MAX_DELAY);

        HAL_UART_Transmit(&huart2,
                    (uint8_t *)rx_msg,
                    strlen(rx_msg),
                    HAL_MAX_DELAY);
      }
    }
}
```